

리눅스 시스템 프로그래밍 학습 기록

남궁명수

[3] 스레드의 기본 개념과 구조.....	2
[01] Thread vs Process: 생성 속도 및 자원 공유 방식의 차이.....	4
[01-1] 프로세스와 스레드의 생성 비용과 메모리 공유 범위는 어떻게 다른가?.....	4
[01-2] 시스템 레벨 관찰: 메모리 맵 분석 및 성능 측정.....	4
[01-3] 이론	7
[01-4] 추가	8
[02] 데이터 레이스(Race Condition): 공유 자원 오염 관찰	10
[02-1] 여러 스레드가 동시에 같은 변수를 수정하면 왜 데이터가 깨지는가?	10
[02-2] 시스템 레벨 관찰: 비결정적 데이터 손실 현상.....	10
[02-3] 이론	12
[02-4] 추가	13
[03] 뮤텍스(Mutex): 임계 구역 보호를 통한 데이터 무결성 보장	14
[03-1] 데이터 오염을 막기 위한 ‘화장실 열쇠(Lock)’메커니즘은 어떻게 동작하는가?. 14	
[03-2] 시스템 레벨 관찰: 동기화 적용 전후의 결과 및 성능 비교.....	14
[03-3] 이론	16
[03-4] 추가	16
[04] 교착 상태(Deadlock): 시스템 마비 상황 재현	19
[04-1] 서로의 자원을 기다리며 영원히 멈춰버리는 상황은 왜 발생하는가?	19
[04-2] 시스템 레벨 관찰: 프로그램 무한 대기(Hang) 및 리소스 점유 상태 분석.....	19
[04-3] 이론	21
[04-4] 추가	21

[3] 스레드의 기본 개념과 구조

스레드를 이해할 때 가장 중요한 출발점은 “스레드는 프로세스 안에서 실제로 실행을 담당하는 흐름이지만, 동시에 그 프로세스의 자원을 공유하는 구조”라는 점이다. 이 두 가지 성질이 동시에 존재하기 때문에 스레드는 성능을 높이는 수단이 되기도하고, 동시에 데이터 오류의 원인이 되기도 한다.

운영체제에서 프로세스는 기본적으로 완전히 독립된 실행 단위이다. 각 프로세스는 자신만의 주소 공간을 가지고 있으며, 다른 프로세스와 메모리를 직접 공유하지 않는다. 그래서 프로세스 간 통신은 IPC 같은 별도의 메커니즘이 필요하다. 반면, 스레드는 같은 프로세스 내부에서 동작하기 때문에 하나의 주소 공간을 공유한다. 이 차이가 스레드의 본질을 결정한다. 스레드는 독립된 실행 흐름이지만, 데이터는 공유된 상태에서 움직이는 구조이다.

이 구조 때문에 스레드는 생성 비용이 매우 낮다. 프로세스를 생성하는 `fork` 는 주소 공간 자체를 복사하거나 `Copy-on-Write` 구조를 통해 새로운 실행 환경을 만들기 때문에 무겁다. 반면, `pthread_create` 는 기존 프로세스의 자원을 그대로 두고 실행 흐름만 추가하는 방식이기 때문에 훨씬 가볍고 빠르다. 그래서 고성능 서버나 병렬 처리가 필요한 시스템에서는 프로세스보다 스레드를 기본 단위로 사용하는 경우가 많다.

하지만 공유 구조는 곧 위험 구조이기도 하다. 스레드는 코드, 데이터, 힙 영역을 공유하기 때문에 하나의 전역 변수나 동적 할당된 메모리를 여러 스레드가 동시에 접근할 수 있다. 여기서 중요한 점은 CPU 입장에서 대부분의 연산이 원자적으로 처리되지 않는다는 것이다. 예를 들어, `count++` 같은 연산은 한 번에 끝나는 것이 아니라 값을 읽고, 더하고, 다시 저장하는 여러 단계로 나뉜다. 이 사이에 다른 스레드가 끼어들면 이전 값이 덮어쓰워지거나 일부 증가 연산이 사라지는 현상이 발생한다. 이것이 데이터 레이스이다.

이 현상은 단순한 논리 오류가 아니라 스케줄링 구조에서 자연스럽게 발생하는 구조이다. 운영체제는 여러 스레드를 빠르게 번갈아 실행하면서 동시에 실행되는 것처럼 보이게 만들기 때문에, 공유 자원에 대한 접근 순서는 예측할 수 없다. 결국 같은 코드라도 실행 타이밍에 따라 결과가 달라지는 비결정성이 발생한다.

이 문제를 해결하기 위해 등장한 개념이 임계 구역이다. 임계 구역은 여러 스레드가 동시에 접근하면 문제가 발생하는 코드 영역을 의미한다. 전역 변수 수정, 파일 쓰기, 데이터베이스 갱신 같은 작업이 여기에 해당한다. 이 영역을 보호하기 위해 사용하는 것이 `mutex` 이다.

`mutex` 는 기본적으로 “한 번에 하나의 스레드만 들어갈 수 있는 잠금 장치” 역할을 한다. 어떤 스레드가 `mutex` 를 획득하면 다른 스레드는 해당 자원에 접근하지 못하고 대기하게 된다. 이를 통해 데이터의 무결성은 보장되지만, 동시에 병렬성은 줄어들고 성능 오버헤드가 발생한다. 즉, 안전성과 성능은 항상 동시에 얻을 수 없는 구조이다.

하지만 mutex 를 잘못 사용하면 더 큰 문제가 발생할 수 있다.

대표적인 것이 데드락이다. 데드락은 여러 스레드가 서로가 가진 자원을 기다리면서 아무도 진행하지 못하는 상태이다.

예를 들어, 스레드 A 가 lock1 을 잡고 lock2 를 기다리고 있는데, 동시에 스레드 B 가 lock2 를 잡고 lock1 을 기다리면 두 스레드는 영원히 서로를 기다리게 된다. 이 상태에서는 CPU 는 아무 작업도 하지 않지만 프로그램은 종료되지 않는다. 시스템 관점에서는 매우 위험한 정지 상태다.

결국 pthread 학습에서 다루는 핵심은 스레드 자체의 사용법이 아니라, “공유 자원을 가진 상태에서 실행 흐름이 동시에 움직일 때 시스템이 어떻게 깨지는가”를 직접 확인하는 것이다.

생성 속도, 메모리 공유, 데이터 레이스, mutex 를 통한 보호, 데드락까지 이어지는 전체 흐름은 하나의 결론으로 이어진다. 스레드는 성능을 극단적으로 높일 수 있는 구조이지만, 동시에 아무 제어 없이 사용하면 시스템의 예측 가능성을 무너뜨리는 구조라는 점이다.

[01] Thread vs Process: 생성 속도 및 자원 공유 방식의 차이

[01-1] 프로세스와 스레드의 생성 비용과 메모리 공유 범위는 어떻게 다른가?

- fork()로 프로세스를 만드는 것과 pthread_create()로 스레드를 만드는 것 중 어느 것이 더 빠른가?
- 또한, 부모 변수 값을 자식 스레드에서 수정하면 부모도 영향을 받는가?

[01-2] 시스템 레벨 관찰: 메모리 맵 분석 및 성능 측정

코드:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <pthread.h>

int global_val = 10;

void *thread_main(void *arg){
    global_val += 5;
    printf("[자식 스레드] 전역 변수 수정 (+5). 현재값: %d, 주소: %p\n", global_val,
(void*)&global_val);
    return NULL;
}

int main(){
    printf("-- [시작] 초기 전역 변수 값: %d, 주소: %p -- \n\n", global_val,
(void*)&global_val);

    printf("[프로세스 시작]\n");
    pid_t pid = fork();

    if(pid == 0){
        global_val += 20;
        printf("[자식 프로세스] 전역 변수 수정 (+20). 현재값: %d, 주소: %p\n",
global_val, (void*)&global_val);
        exit(0);
    }
    else{
        wait(NULL);
    }
}
```

```

        printf("[부모 프로세스] 자식 종료 후 현재값: %d, 주소: %p\n", global_val,
(void*)&global_val);
    }

    printf("[스레드 시작]\n");
    pthread_t tid;

    if(pthread_create(&tid, NULL, thread_main, NULL) != 0){
        perror("thread 생성 실패");
        return 1;
    }
    pthread_join(tid, NULL); // 스레드가 끝날 때까지 대기
    printf("[메인 스레드] 자식 스레드 종료 후 현재값: %d, 주소: %p\n", global_val,
(void*)&global_val);

    return 0;
}

```

결과:

```

gcc -g 01_thread_vs_process.c -o 01.out -lpthread
time ./01.out

```

표준 출력은 아래와 같다.

```

-- [시작] 초기 전역 변수 값: 10, 주소: 0x55b7a0678010 -
[프로세스 시작]
[자식 프로세스] 전역 변수 수정 (+20). 현재값: 30, 주소: 0x55b7a0678010
[부모 프로세스] 자식 종료 후 현재값: 10, 주소: 0x55b7a0678010

real 0m0.003s
user 0m0.001s
sys 0m0.002s

```

```

-- [시작] 초기 전역 변수 값: 10, 주소: 0x5587c0648010 -
[스레드 시작]
[자식 스레드] 전역 변수 수정 (+5). 현재값: 15, 주소: 0x5587c0648010
[메인 스레드] 자식 스레드 종료 후 현재값: 15, 주소: 0x5587c0648010

real 0m0.002s
user 0m0.002s

```

결과 해석:

먼저 fork 구간을 보면 초기 값은 10 이고 자식 프로세스에서 +20 을 해서 30 이 된다.

여기까지는 자식 프로세스내부에서 정상적으로 변경된 것이다.

하지만 중요한 것은 부모 프로세스에서는 여전히 값이 10 으로 유지된다는 점이다.

출력상으로는 두 프로세스가 동일한 주소를 사용하고 있기 때문에 마치 같은 메모리를 공유하는 것처럼 보이지만, 실제로는 CoW 구조 때문에 메모리가 분리되어 있다.

즉, fork() 시점에서는 같은 가상 주소를 유지하지만, 쓰기 연산이 발생하는 순간 물리 메모리가 분리되기 때문에 자식 프로세스의 변경은 부모에게 전혀 영향을 주지 않는다.

그래서 부모는 자식 종료 이후에도 값이 그대로 10 으로 유지된다.

이 구조가 바로 프로세스의 핵심 특징인 “격리된 실행 환경”이다.

이 결과에서 중요한 것은 주소가 같다는 정보는 실제 공유를 의미하지 않는다는 점이다.

운영체제는 프로세스마다 독립된 주소 공간을 유지하지만, 사용자 입장에서는 동일한 가상 주소를 보여주기 때문에 혼동이 발생한다. 그러나 실제 메모리 관점에서는 완전히 분리된 상태이기 때문에 상태 공유가 발생하지 않는다.

다음으로 스레드 구간을 보면 초기값 10 에서 +5 가 발생하여 15 가 되고, 이 값이 메인 스레드에도 그대로 반영된다.

여기서 중요한 차이는 주소가 같다는 수준이 아니라 실제 메모리 자체를 공유하고 있다는 점이다. 동일한 프로세스 내부에서 동작하는 스레드들은 코드, 데이터, 힙 영역을 모두 공유하기 때문에 global_val 은 하나의 실체를 가진다.

따라서 자식 스레드에서 값을 변경하면 그 변경은 즉시 다른 스레드에서도 관측된다.

그래서 결과적으로 자식 스레드가 15 를 출력하고, 메인 스레드도 동일하게 15 를 출력한다. 이는 단순한 출력 일치가 아니라 하나의 메모리 공간을 여러 실행 흐름이 동시에 참조하고 있다는 구조적 증거이다.

따라서, fork는 “같은 주소처럼 보이지만 실제로는 분리된 메모리 구조”이고, pthread는 “같은 주소이면서 실제 메모리도 공유하는 구조”이다.

time 결과도 같은 맥락에서 설명된다.

fork 와 pthread 모두 매우 짧은 시간 안에 실행되지만, 이 수치는 성능 차이를 보여주는 것이 아니라, 각각의 생성 및 스케줄링 방식이 얼마나 가벼운지를 보여주는 보조 데이터다.

real 값은 전체 경과 시간, user 는 사용자 코드 실행 시간, sys 는 커널이 프로세스나 스레드를 관리하는 데 사용한 시간인데, 이 학습에서는 fork 와 pthread 모두 매우 작은 값을 가지기 때문에 구조적 차이를 해석하는데 있어 핵심 지표는 아니다.

[01-3] 이론

(1) LWP(Light Weight Process)와 Shared Address Space

스레드를 운영체제 관점에서 이해할 때 중요한 개념이 LWP 이다.

이는 “스레드는 완전히 별도의 개념이 아니라, 커널 내부에서 프로세스와 거의 동일한 단위로 관리되지만 일부 자원을 공유하는 경량 프로세스”라는 구조를 설명하기 위한 모델이다.

리눅스 커널에서는 프로세스와 스레드를 완전히 다른 구조로 구분하지 않고, 모두 `task_struct` 라는 동일한 자료구조로 관리된다. 즉, 커널 입장에서는 “프로세스”와 “스레드”를 본질적으로 같은 실행단위로 취급한다. 다만 어떤 자원을 공유하느냐에 따라 동작 방식이 달라지며, 이 차이를 통해 스레드가 구현된다.

스레드는 생성될 때 새로운 `task_struct` 를 가지지만, 메모리 관리 정보인 `mm_struct` 를 부모와 공유한다. 이 `mm_struct` 는 프로세스의 주소 공간, 즉 코드 영역, 데이터 영역, 힙 영역 등의 메모리 구조를 관리하는 핵심 구조체이다.

따라서 같은 `mm_struct` 를 공유한다는 것은 곧 같은 주소 공간을 공유한다는 의미가 된다.

이 구조 때문에 스레드는 서로 독립된 실행 흐름을 가지면서도 동일한 메모리 공간을 참조할 수 있다. 즉, 각 스레드는 별도의 실행 상태(레지스터, 스택 등)는 가지고 있지만, 데이터 영역은 공유하는 형태가 된다. 이 때문에 전역 변수나 힙 영역 데이터는 모든 스레드에서 동일하게 접근되고 변경될 수 있다.

결과적으로 LWP 는 “프로세스처럼 스케줄링되지만, 프로세스 내부 자원을 공유하는 실행 단위”라고 정의할 수 있다. 커널은 이를 통해 스레드를 별도의 무거운 프로세스로 생성하지 않고도 빠르게 생성하고 스케줄링할 수 있으며, 동시에 하나의 프로세스 내부에서 여러 실행 흐름이 동작하는 구조를 제공한다.

이 구조는 성능 측면에서는 매우 효율적이지만, 메모리 공유로 인해 데이터 레이스나 동기화 문제가 발생할 수 있는 근본적인 원인이 된다.

[01-4] 추가

(1) fork()와 pthread_create()의 strace 비교 분석

strace 를 이용하면 fork()와 pthread_create()는 모두 커널 내부적으로 clone() 계열 시스템 콜로 구현된다는 것을 확인할 수 있다. 그러나 두 방식은 전달되는 플래그의 구성에서 명확한 차이를 보이며, 이는 곧 프로세스와 스레드의 구조적 차이로 이어진다.

fork clone 호출 결과:

```
clone(child_stack=NULL,
flags=CLONE_CHILD_CLEAR_TID|CLONE_CHILD_SETTID|SIGCHLD,
child_tidptr=0x7f6d2bdeba10) = 46205
```

fork()의 경우 clone() 호출 시 CLONE_VM 플래그가 포함되어 있지 않다.
이 점이 가장 중요한 차이점이다.

- 부모 프로세스와 자식 프로세스는 주소 공간을 공유하지 않는다.
- 커널은 새로운 프로세스를 생성하면서 별도의 주소 공간을 구성한다.
- 페이지 테이블 구조가 복제되며, 필요 시 CoW 방식으로 물리 메모리가 분리된다.
- 결과적으로 완전히 독립된 프로세스가 생성된다.

또한, SIGCHLD 플래그는 자식 프로세스 종료 시 부모 프로세스에게 신호를 전달하기 위한 기본 동작이다. 즉, fork()는 새로운 프로세스를 생성하는 방식이며, 메모리 구조가 분리되는 것이 핵심 특징이다.

pthread_create()의 clone3 호출:

```
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEAR_TID, child_tid=0x7ffaf99c7910, parent_tid=0x7ffaf99c7910, exit_signal=0, stack=0x7ffaf91c7000, stack_size=0x7fff00, tls=0x7ffaf99c7640})strace: Process 46657 attached
=> {parent_tid=[46657]}, 88) = 46657
```

pthread_create()의 경우 CLONE_VM 플래그가 포함되어 있다. 이는 fork()와 가장 중요한 차이점이다.

각 플래그의 의미는 다음과 같다.

- CLONE_VM: 부모와 스레드가 동일한 가상 메모리 공간을 공유
- CLONE_FILES: 파일 디스크립터 공유
- CLONE_SIGHAND: signal handler 를 공유
- CLONE_THREAD: 동일한 프로세스 그룹 내 스레드로 동작
- CLONE_SETTLS: thread local storage 설정

또한 스레드별로 독립적인 stack 영역은 별도로 할당된다.

즉, `pthread_create()`는 새로운 프로세스를 생성하는 것이 아니라, 기존 프로세스 내부에서 실행 흐름을 추가하는 방식이다.

(2) 결론:

starce 관점에서 `fork()`와 `pthread_create()`의 차이는 `clone()` 호출 시 전달되는 플래그 구성에서 명확하게 드러난다.

- `fork()`는 `CLONE_VM`이 없기 때문에 독립적인 주소 공간을 가지는 프로세스를 생성한다.
- `pthread_create()`는 `CLONE_VM`이 존재하여 동일한 주소 공간을 공유하는 스레드를 생성한다.

이 차이는 곧 프로세스와 스레드의 본질적인 구조 차이를 반영한다.

(3) 스레드 간 격리 부재와 정보 유출 위험

스레드는 동일한 프로세스 내부에서 실행되며 코드 영역, 데이터 영역, 힙 영역을 포함한 주소 공간을 공유한다. 이러한 구조는 성능 측면에서는 매우 효율적이지만, 보안 관점에서는 근본적인 한계를 가진다.

즉, 하나의 프로세스 내부에서는 실행 흐름만 분리되어 있을 뿐, 데이터는 완전히 분리되지 않는다.

이 구조에서 문제가 되는 상황은 특정 스레드가 취약점이나 공격에 의해 제어권을 탈취하는 경우이다. 스레드는 독립된 메모리 공간을 가지지 않기 때문에, 공격자가 해당 스레드를 통해 같은 프로세스 내 다른 스레드가 사용 중인 메모리 영역에 접근할 수 있는 가능성이 발생한다. 이때 접근 가능한 범위에는 전역 변수뿐만 아니라 힙 영역에 저장된 데이터도 포함되며, 이는 사실상 프로세스 전체 메모리에 대한 접근과 동일한 수준의 위험을 의미한다.

이로 인해 암호 키, 세션 정보, 인증 토큰과 같은 보안적으로 민감한 데이터가 함께 노출될 수 있으며, 이는 단일 스레드의 침해가 전체 애플리케이션 보안으로 확장되는 구조적 취약점으로 이어진다. 특히, 웹 서버나 보안 엔진처럼 하나의 프로세스 안에서 여러 요청을 스레드로 처리하는 구조에서는 이러한 위험이 더욱 커진다.

반면 프로세스는 기본적으로 독립된 주소 공간을 가지며 서로의 메모리에 직접 접근할 수 없도록 격리되어 있다. 따듯 동일한 취약점이 발생하더라도 영향 범위가 해당 프로세스 내부로 제한된다. 이 때문에 보안이 중요한 시스템에서는 성능 효율보다 격리성을 우선하여 프로세스 단위 분리를 사용하는 경우가 많다.

결론적으로 스레드 기반 구조는 성능과 자원 효율성 측면에서는 유리하지만, 보안 측면에서는 하나의 취약점이 전체 메모리 노출로 이어질 수 있는 구조적 위험을 가진다. 따라서 민감한 데이터를 다루는 환경에서는 스레드보다 프로세스 기반 격리 구조가 더 안전한 설계로 간주된다.

[02] 데이터 레이스(Race Condition): 공유 자원 오염 관찰

[02-1] 여러 스레드가 동시에 같은 변수를 수정하면 왜 데이터가 깨지는가?

- 전역 변수 count 를 2 개의 스레드가 각각 10 만 번씩 더했을 때, 결과는 정확히 20 만이 나오는가?

[02-2] 시스템 레벨 관찰: 비결정적 데이터 손실 현상

코드:

```
#include <stdio.h>
#include <pthread.h>

#define ITERATIONS 100000

long long shared_count = 0;

void *thread_func(void *arg){
    for(int i = 0; i < ITERATIONS; i++){
        shared_count++; // 3번의 동작으로 나뉘서 연산됨
    }
    return NULL;
}

int main(){
    printf("[시작] 초기값: %lld, 목표값: %dWn", shared_count, ITERATIONS*2);

    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, thread_func, NULL);
    pthread_create(&thread2, NULL, thread_func, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("최종 결과: %lldWn", shared_count);

    if(shared_count == ITERATIONS * 2){
        printf("결과: 정확Wn");
    }
    else{
        printf("결과: 데이터 유실(차이 %lld)Wn", (ITERATIONS*2) - shared_count);
    }
}
```

```
}

return 0;
}
```

결과:

표준 출력은 다음과 같다.

[시작] 초기값: 0, 목표값: 200000

최종 결과: 93852

결과: 데이터 유실(차이 106148)

결과 해석:

이 학습의 최종 결과 93852는 기대값인 200000에 도달하지 못한 것으로, 약 106148개의 증가 연산이 유실된 상태이다. 이는 연산 로직의 오류가 아니라, 두 개의 스레드가 동일한 전역 변수 `shared_count`에 대해 동기화 없이 동시에 접근하면서 발생한 전형적인 데이터 레이스(Race Condition) 결과이다.

이 결과는 스레드가 많아져서 생긴 “성능 문제”가 아니라, 하나의 공유 자원에 대해 원자성이 보장되지 않은 연산이 동시에 실행되었기 때문에 발생한 구조적 오류이다. 즉, `shared_count++` 연산이 실행되는 과정에서 일부 증가 연산이 서로 덮어쓰기 되면서 사라진 것이다.

이 코드에서 두 개의 스레드는 각각 100000번씩 `shared_count++`를 수행하므로, 정상적인 경우라면 최종 값은 200000이 되어야 한다. 그러나 실제 연산은 다음과 같이 분리되어 수행된다.

- `shared_count` 값을 읽는다(Load)
- 1 증가시킨다(Add)
- 결과를 저장한다(Store)

이 과정은 하나의 원자적 연산이 아니라 여러 단계로 나뉘어 있기 때문에, 두 스레드가 동시에 실행되면 중간 상태에서 값이 서로 덮어써질 수 있다.

예를 들어,

- 스레드 A와 B가 동시에 `shared_count = 0`을 읽음
- A가 1을 저장
- B가 1을 저장

이 경우 증가 연산 1회가 사라진다. 이러한 충돌이 반복되면서 누적적으로 큰 값의 손실이 발생하게 된다.

이 결과는 멀티스레드 환경에서 공유 자원에 대한 접근이 동기화되지 않을 경우, 실행 결과가 논리적으로 보장되지 않는다는 것을 보여준다. 특히, 스레드 스케줄링은 운영체제에 의해 비결정적으로 이루어지기 때문에, 실행할 때마다 손실되는 값의 크기는 달라질 수 있다.

따라서 이 학습의 본질은 단순히 “값이 틀렸다”가 아니라, 공유 메모리 + 비원자적 연산 + 동시 실행 = 데이터 유실이라는 구조적 관계를 확인하는 것이다.

[02-3] 이론

(1) 원자성(Atomicity)과 경쟁 상태(Race Condition)

count++ 연산은 고급 언어 수준에서 단일 연산처럼 보이지만, 실제 CPU 및 어셈블리 수준에서는 하나의 원자적(atomic)으로 연산으로 처리되지 않고 여러 단계로 분리되어 수행된다. 일반적으로 해당 연산은 다음과 같은 순서로 실행된다.

먼저 메모리에서 현재 count 값을 읽어오는 Load 단계가 수행되고, 이후에 해당 값에 1을 더하는 Add 연산이 진행되며, 마지막으로 증가된 값을 다시 메모리에 저장하는 Store 단계가 실행된다. 즉, count++는 하나의 명령이 아니라 최소 3단계 이상의 분리된 연산으로 구성된다.

이 구조에서 중요한 점은 각 단계 사이에 실행 중단(interruption)이 발생할 수 있다는 것이다. 운영체제는 스레드를 비결정적인 시점에서 선점(preemption)할 수 있으며, 이 과정에서 컨텍스트 스위칭이 발생하면 다른 스레드가 동일한 변수에 접근하게 된다. 이때 두 스레드가 같은 값을 기반으로 연산을 수행하면 서로의 결과를 덮어쓰게 되는 상황이 발생할 수 있다.

예를 들어, 두 스레드가 동시에 count 값을 0으로 읽은 경우, 각각 1을 증가시켜 저장하게 되면 최종 결과는 2가 아니라 1이 된다.

이는 한 번의 증가 연산이 손실된 결과이며, 이러한 현상이 반복되면 기대한 값과 실제 값 사이에 큰 차이가 발생하게 된다.

이처럼 여러 실행 흐름이 공유 자원에 동시에 접근하면서 발생하는 비결정적인 결과 문제를 경쟁 상태(Race Condition)라고 한다. Race Condition의 본질은 연산 자체의 오류가 아니라, 연산이 원자적으로 수행되지 않는 상태에서 실행 순서가 보장되지 않는다는 점에 있다.

따라서 원자성은 하나의 연산이 중간에 끊기지 않고 완전히 수행되거나 전혀 수행되지 않는 성질을 의미하며, count++는 기본적으로 이 원자성을 보장하지 않기 때문에, 별도의 동기화 없이 멀티스레드 환경에서 사용될 경우 데이터 유실이 발생할 수 있다.

[02-4] 추가

(1) TOCTOU(Time-of-Check to Time-of-Use)취약점

보안 시스템에서 발생하는 대표적인 동시성 기반 취약점 중 하나가 TOCTOU 이다.
이는 Race Condition 을 악용한 공격 방식으로, “검사 시점과 실제 사용 시점 사이의 시간 차이”를 이용해 시스템의 판단을 무력화하는 구조이다.

이 취약점은 다음과 같은 단계로 발생한다.

먼저, Check 시점에서 프로그램은 특정 자원에 대한 접근 권한을 검사한다. 예를 들어, “이 사용자가 해당 파일을 읽을 수 있는 권한이 있는가”를 확인하는 단계이다.

이 시점에서는 시스템이 정상적인 상태를 기준으로 판단을 수행한다.

그 다음 Race 시점에서는 검사와 실제 사용 사이의 아주 짧은 시간 간격이 존재한다.

이 구간은 시스템 입장에서는 정상적인 흐름처럼 보이지만, 공격자는 이 찰나의 시간에 개입할 수 있다. 예를 들어, 공격자는 해당 파일을 권한이 낮은 일반 파일에서 관리자 권한이 필요한 민감한 파일로 바꿔치기하는 방식으로 상태를 변경한다.

마지막 Use 시점에서는 이미 권한 검사가 끝난 상태이기 때문에 프로그램은 “안전하다”고 판단된 대상에 대해 작업을 수행한다. 그러나 실제로는 공격자가 변경한 파일이 대상이 되어 있기 때문에, 의도하지 않은 높은 권한의 자원에 접근하게 된다.

이 구조의 핵심 문제는 Check 와 Use 가 분리된 상태로 존재한다는 점이다.

즉, 시스템이 “검사 결과를 신뢰하는 순간”과 “실제 작업을 수행하는 순간” 사이에 상태 변화가 발생할 수 있기 때문에, 논리적으로 안전하다고 판단된 결과가 실제 실행 시점에서는 더 이상 유효하지 않을 수 있다.

따라서 보안 시스템에서는 이러한 문제를 근본적으로 방지하기 위해 Check 와 Use 사이에 어떤 외부 개입도 발생하지 않도록 해야 하며, 이를 위해 원자성을 보장하는 구조가 필요하다.

즉, 검사와 사용을 하나의 연산처럼 묶어서 중간 상태가 존재하지 않도록 설계해야 TOCTOU 취약점을 방지할 수 있다.

[03] 뮤텝스(Mutex): 임계 구역 보호를 통한 데이터 무결성 보장

[03-1] 데이터 오염을 막기 위한 ‘화장실 열쇠(Lock)’메커니즘은 어떻게 동작하는가?

- 공유 자원에 접근할 때 한 번에 한 명만 들어갈 수 있게 제한하면 결과가 정확해지는가?

[03-2] 시스템 레벨 관찰: 동기화 적용 전후의 결과 및 성능 비교

코드:

```
#include <stdio.h>
#include <pthread.h>
#define ITERATIONS 100000

long long shared_count = 0;
pthread_mutex_t lock;

void *thread_func(void *arg){
    for(int i = 0; i < ITERATIONS; i++){

        pthread_mutex_lock(&lock);
        shared_count++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main(){
    printf("[시작] 초기값: %lld\n", shared_count);

    pthread_t thread1, thread2;

    pthread_create(&thread1, NULL, thread_func, NULL);
    pthread_create(&thread2, NULL, thread_func, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    // 뮤텝스 제거
    pthread_mutex_destroy(&lock);

    printf("최종 결과: %lld\n", shared_count);
}
```

```
if(shared_count == ITERATIONS * 2){
    printf("결과: 정확Wn");
}
else {
    printf("결과: 오염 %lldWn", ITERATIONS * 2 - shared_count);
}

return 0;
}
```

결과:

표준 출력은 다음과 같다.

```
[시작] 초기값: 0
최종 결과: 200000
결과: 정확
```

결과 해석:

이 프로그램의 실행 결과가 항상 정확하게 200000 으로 나오는 이유는 pthread_mutex_lock 과 pthread_mutex_unlock 을 이용해 공유 변수 shared_count 에 대한 접근을 하나의 임계 구역으로 보호했기 때문이다.

두 개의 스레드가 동시에 shared_count++ 연산을 100000 번 수행하지만, 실제 실행 과정에서는 동시에 접근하는 구조가 아니라 항상 “하나의 스레드만 접근 가능한 상태”로 직렬화되어 처리된다.
구체적으로 한 스레드가 pthread_mutex_lock(&lock)을 호출하여 뮤텍스를 획득하면, 해당 스레드만 임계 구역에 진입할 수 있고 다른 스레드는 락이 해제될 때까지 해당 위치에 대기 상태에 머물게 된다.

이 상태에서 락을 획득한 스레드는 shared_count++연산을 수행한다.
이 연산은 내부적으로 Load, Add, Store 의 3 단계로 구성된 비원자적 연산이지만, 뮤텍스에 의해 외부 스레드의 개입이 완전히 차단되기 때문에 실행 중간에 다른 스레드가 끼어들 수 없다.
이후 연산이 완료되면 pthread_mutex_unlock(&lock)이 호출되며, 그제서야 다른 스레드가 임계 구역에 진입할 수 있는 권한을 얻는다.

이 과정이 반복되면서 두 스레드는 동일한 공유 자원에 대해 동시에 접근하지 않고 순차적으로 실행된다. 따라서 race condition 이 발생할 여지가 사라지고, 모든 증가 연산이 손실 없이 정확하게 반영된다. 결과적으로 두 스레드가 수행한 총 20000 회의 증가 연산이 그대로 유지되어 최종 값이 정확하게 20000 으로 출력된다.

[03-3] 이론

상호 배제(Mutual Exclusion)와 임계 구역(Critical Section)

이 학습은 멀티스레드 환경에서 발생하는 동기화 문제를 해결하는 대표적인 구조를 보여준다.

임계 구역은 둘 이상의 스레드가 동시에 접근하면 데이터 불일치가 발생하는 코드 영역으로, 코드에서는 `shared_count++`가 해당된다. 이 영역은 반드시 한 번에 하나의 스레드만 접근할 수 있도록 제한되어야 한다.

이를 보장하는 개념이 상호 배제이며, 이는 특정 스레드가 공유 자원을 사용하는 동안 다른 스레드의 접근을 차단하는 원리이다. `pthread_mutex_lock` 과 `pthread_mutex_unlock` 은 이러한 상호 배제를 구현하는 대표적인 동기화 도구이다.

또한, 이 구조는 결과적으로 실행 흐름을 병렬이 아닌 직렬화된 형태로 변환한다. 즉, 여러 스레드가 존재하더라도 임계 구역 내부에서는 한 번에 하나의 스레드만 실행되기 때문에, 해당 구간은 논리적으로 단일 스레드처럼 동작하게 된다.

[03-4] 추가

(1) Thread-Safe(스레드 안전)

Thread-Safe 란 여러 스레드가 동시에 동일한 자원에 접근하더라도 데이터의 일관성과 정확성이 보장되는 상태를 의미한다. 즉, 동시 실행 환경에서도 결과가 깨지지 않고 항상 예측 가능한 형태로 유지되는 구조를 말한다.

보안 엔진과 같은 시스템에서는 여러 스레드가 동시에 동작하는 것이 일반적이다. 예를 들어, 수백 개의 스레드가 동시에 파일을 검사하면서 악성 코드를 탐지하고, 그 결과를 로그 파일에 기록하는 상황을 생각할 수 있다. 이때 로그 파일은 모든 스레드가 공유하는 단일 자원이 되기 때문에, 적절한 동기화 없이 동시에 `write`가 발생하면 데이터가 서로 섞이거나 일부가 덮어쓰지는 문제가 발생한다. 그 결과 로그는 정상적인 기록이 아니라 의미를 해석할 수 없는 깨진 데이터로 변형될 수 있다.

이러한 문제는 공유 자원에 대한 동시 접근이 제어되지 않았기 때문에 발생하며, 이를 방지하기 위해 Mutex 와 같은 동기화 기법이 사용된다. Mutex 를 통해 한 시점에 하나의 스레드만 로그 파일에 접근하도록 제한하면 데이터 충돌을 방지할 수 있고, 결과적으로 로그의 일관성과 정확성이 보장된다.

하지만 Thread-Safe 를 확보하는 과정에는 항상 성능과의 trade-off 가 존재한다. 동기화를 강화하면 데이터의 정확성은 높아지지만, 스레드 간 대기 시간이 증가하면서 전체 처리 속도는 감소한다. 반대로 성능을 높이기 위해 동기화를 최소화하면 race condition 과 데이터 손상이 발생할 가능성이 증가한다. 즉, 보안성과 성능은 동시에 극대화하기 어려운 구조적 관계에 있다.

이러한 문제를 해결하기 위해 단순히 전체 구간을 락으로 보호하는 방식이 아니라, 락의 범위를 최소화하는 Fine-grained Locking 기법을 사용한다. 이는 전체 자원을 하나의 락으로 보호하는 대신, 필요한 부분만 작은 단위로 나누어 락을 적용하여 병렬성을 최대한 유지하는 방식이다.

또한, 최근에는 락 자체를 사용하지 않는 Lock-free 자료구조나 알고리즘도 연구 및 적용되고 있다. 이는 CAS(Compare-And-Swap)와 같은 원자적 연산을 기반으로 하여 동기화없이 데이터 일관성을 유지하는 방식이다. 이러한 접근은 성능과 안전성을 동시에 확보하기 위한 고급 최적화 기법으로, 대규모 실시간 탐지 시스템에서 중요한 설계 방향으로 활용된다.

(2) CAS(Compare-And-Swap, 비교 후 교환)

멀티스레드 환경에서 락 없이 동기화를 구현하는 대표적인 원자적 연산이다.

“값을 바꾸기 전에 현재 값이 내가 예상한 값인지 확인하고, 맞을 때만 변경한다”는 것이다.

기본 동작 구조

- 메모리 주소(value)
- 기대값(expected)
- 새로운 값(new value)

동작 방식은 다음과 같다.

- 현재 메모리 값이 expected 와 같으면 -> new value 로 변경
- 같지 않으면 -> 아무것도 하지 않고 실패

이 과정은 CPU 레벨에서 하나의 원자적 명령으로 실행된다.

동작 예시

예를 들어 공유 변수 count = 10 이 있다고 가정하자.

스레드 A 의 CAS

- expected = 10
- new value = 11

CAS 실행

- 현재 count == 10 -> 같음
- 따라서 count 를 11 로 변경

스레드 B 의 CAS(동시에 실행)

- expected = 10

- new value = 20

하지만 이미 A 가 변경했기 때문에

- 현재 count == 1 -> expected(10)과 다름

- 변경 실패

즉, B 는 다시 시도해야 한다.

```
int compare_add_swap(int *addr, int expected, int new_val){
    if(*addr == expected) {
        *addr = new_val;
        return 1;
    }
    return 0;
}
```

```
void *thread_func(void *arg){
    for(int i = 0; i < 100000; i++){
        int old;

        do {
            old = shared_count;
        } while(!compare_and_swap(&shared_count, old, old+1));
    }
}
```

[04] 교착 상태(Deadlock): 시스템 마비 상황 재현

[04-1] 서로의 자원을 기다리며 영원히 멈춰버리는 상황은 왜 발생하는가?

- 두 개의 스레드가 두 개의 락을 서로 엇갈린 순서로 잡으려고 하면 어떤 일이 벌어지는가?

[04-2] 시스템 레벨 관찰: 프로그램 무한 대기(Hang) 및 리소스 점유 상태 분석

코드:

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

// 두 개의 자원 준비
pthread_mutex_t lock1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lock2 = PTHREAD_MUTEX_INITIALIZER;

void *thread_A(void *arg){
    printf("[스레드 A] 1번 열쇠 잡기");
    pthread_mutex_lock(&lock1);
    printf("[스레드 A] 1번 열쇠 획득: 2번 열쇠 잡으러 가기");

    sleep(1);

    pthread_mutex_lock(&lock2);
    printf("[스레드 A] 2번 열쇠까지 획득Wn");

    pthread_mutex_unlock(&lock2);
    pthread_mutex_unlock(&lock1);
    return NULL;
}

void *thread_B(void *arg){
    printf("[스레드 B] 2번 열쇠 잡기");
    pthread_mutex_lock(&lock2);
    printf("[스레드 B] 2번 열쇠 획득: 1번 열쇠 잡으러 가기");

    sleep(1);

    pthread_mutex_lock(&lock1);
    printf("[스레드 B] 1번 열쇠까지 획득Wn");
```

```

    pthread_mutex_unlock(&lock1);
    pthread_mutex_unlock(&lock2);
    return NULL;
}

int main(){
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread_A, NULL);
    pthread_create(&t2, NULL, thread_B, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("프로그램이 정상 종료\n");
    return 0;
}

```

결과:

무한 대기

결과 해석:

이 프로그램이 실행되면 스레드 A와 스레드 B는 각각 서로 다른 순서로 두 개의 뮤텍스(lock1, lock2)를 획득하려고 시도한다. 문제의 핵심은 두 스레드가 동일한 자원을 서로 반대 순서로 점유하려고 한다는 점이다.

스레드 A는 먼저 lock1을 획득한 뒤 lock2를 기다리는 구조이고, 스레드 B는 반대로 lock2를 먼저 획득한 뒤 lock1을 기다리는 구조이다. 이 상태에서 각각 sleep(1)을 통해 스레드가 먼저 자원을 획득할 시간을 의도적으로 만들어 주기 때문에, 두 스레드는 동시에 “서로 가진 자원을 기다리는 상태”에 진입하게 된다.

결과적으로 스레드 A는 lock1을 점유한 채 lock2를 기다리고, 스레드 B는 lock2를 점유한 채 lock1을 기다리면서 어느 쪽도 진행할 수 없는 상태가 된다.

이 상태에서는 CPU가 어떤 연산도 수행하지 않지만, 프로그램은 종료되지 않고 무한 대기 상태에 빠지게 된다.

이는 단순한 지연이 아니라 실행 흐름 자체가 교착되어 더 이상 진행이 불가능한 상태이다.

또한 top이나 htop으로 확인하면 CPU 사용률은 거의 0%에 가깝게 나타나지만 프로세스는 종료되지 않는다. 이는 무한 루프가 아니라 스레드가 모두 lock 획득 지점에서 block 상태로 멈춰 있기 때문이다. 즉, CPU를 사용하지 않고 “기다림 상태”에 갇혀 있는 구조이다.

[04-3] 이론

이 상황은 데드락(Deadlock)의 전형적인 발생 조건을 모두 만족하는 구조이다.

먼저 상호배제는 하나의 뮤텁스를 동시에 하나의 스레드만 사용할 수 있다는 특성으로 인해 발생한다. lock1 과 lock2 는 동시에 두 스레드가 점유할 수 있다.

두 번째는 점유 및 대기(Hold and Wait)로, 각 스레드는 이미 하나의 락을 점유한 상태에서 다른 락을 기다리고 있다.

세 번째는 비선점(No Preemption)으로, 이미 점유된 락은 외부에서 강제로 빼앗을 수 없기 때문에 각 스레드는 스스로 락을 해제하기 전까지 유지된다.

마지막으로 순환 대기는 스레드 A 와 B 가 서로의 락을 기다리는 구조로 인해 발생하며, 이로 인해 대기 관계가 원형 구조를 이루게 된다.

이 네 가지 조건이 동시에 만족되면 시스템은 더 이상 진행할 수 없는 데드락 상태에 빠지게 된다.

- 상호배제
- 점유 및 대기
- 비선점
- 순환 대기

[04-4] 추가

Dos 공격과 데드락

데드락은 단순한 프로그램 오류나 설계 실수로 끝나는 문제가 아니라, 보안 시스템에서는 서비스 전체를 무력화시킬 수 있는 구조적 취약점으로 이어질 수 있다. 특히, 실시간 보안 엔진에서는 데드락이 발생하는 순간 시스템이 종료되지 않고, “동작은 하지만 멈춘 상태”가 될 수 있기 때문에, 이는 사실상 서비스 거부(DoS)와 동일한 효과를 가진다.

먼저 Denial of Service(DoS)의 관점에서 보면, 보안 엔진이 정상적으로 실행 중이더라도 내부 스레드가 데드락 상태에 빠지면 핵심 기능은 동작하지 않게 된다.

즉, 프로세스 자체는 살아 있지만, 파일 검사, 실시간 탐지, 로그 기록과 같은 보안 기능이 모두 멈추게 된다. 이 상태에서는 외부에서 들어오는 악성코드나 공격을 더 이상 탐지하거나 차단할 수 없기 때문에, 시스템은 “켜져 있지만 방어 기능이 없는 상태”가 된다.

이는 공격자 입장에서 가장 이상적인 침투 환경이다.

공격 시나리오 관점에서는 데드락의 의도적으로 유도될 수 있다는 점이 중요하다.

예를 들어, 공격자는 보안 엔진이 특정 자원에 접근할 때 사용하는 락 순서나 파일 접근 패턴을 분석한 뒤, 이를 역이용하여 순환 대기가 발생하도록 유도할 수 있다. 즉, 특정 파일

구조나 요청 순서를 반복적으로 조작하여 스레드들이 서로 다른 락을 점유한 채 서로를 기다리게 만드는 방식으로 시스템을 정지 상태에 빠뜨릴 수 있다.

이러한 공격은 단순히 CPU 를 과도하게 사용하는 방식이 아니라, 오히려 CPU 사용률을 낮춘 상태에서 시스템을 멈추게 만드는 특징이 있다. 따라서 일반적인 성능 모니터링만으로는 탐지하기 어렵고, 보안적으로 더 위험한 형태의 DoS 로 분류된다.

이 문제를 방지하기 위한 엔지니어링 원칙은 명확하다.

가장 기본적인 원칙은 락 계층 구조를 설계하는 것이다. 모든 스레드가 동일한 순서로 락을 획득하도록 강제하면 순환 대기 조건이 발생하지 않기 때문에 데드락 자체를 구조적으로 차단할 수 있다.

또한 락의 범위를 가능한 한 최소화하는 것도 중요한 설계 요소이다.

임계 구역이 길어질수록 락 점유 시간이 증가하고, 그만큼 데드락 발생 가능성도 높아진다. 따라서 필요한 최소 구간만 락으로 보호하는 방식이 권장된다.

추가적으로 pthread_mutex_trylock 과 같은 비블로킹 방식의 락을 활용하면 무한 대기를 방지할 수 있다. 이는 락 획득에 실패했을 때 즉시 다른 로직으로 분기할 수 있도록 하여, 시스템이 영구적으로 멈추는 상황을 예방하는 역할을 한다.

결론적으로 데드락은 단순한 동시성 오류가 아니라, 보안 시스템 전체를 마비시킬 수 있는 서비스 거부 취약점으로 이어질 수 있다. 따라서 안정적인 보안 엔진 설계를 위해서는 락 순서 설계, 임계 구역 최소화, 비동기/비블로킹 락 전략이 함께 고려되어야 한다.